



Hệ thống giám sát tài xế thông minh tích hợp sáu module thị giác máy tính phát hiện hành vi thời gian thực

Đào Văn Vinh^{1,*}, Đào Phúc Dân¹, Nguyễn Thanh Xuân¹

¹Khoa Công nghệ Thông tin, Trường Đại học Công nghệ Giao thông vận tải, Hà Nội, Việt Nam

Thông tin bài viết

Dạng bài viết:

Bài báo nghiên cứu khoa học sinh viên

*Tác giả liên hệ:

Địa chỉ E-mail:

daovinhgm2005@gmail.com

Giảng viên hướng dẫn:

Đoàn Thị Thanh Hằng¹

Giảng viên nhận xét:

Phạm Đức Anh¹

Tóm tắt: Buồn ngủ, mất tập trung và vi phạm quy tắc lái xe là những nguyên nhân hàng đầu gây thương vong giao thông tại Việt Nam. Bài báo đề xuất một hệ thống giám sát tài xế thông minh dựa trên trí tuệ nhân tạo, tích hợp 6 module thị giác máy tính bao gồm: phát hiện buồn ngủ, ngáp, sử dụng điện thoại, không thắt dây an toàn, tư thế đầu và vị trí tay lái. Hệ thống sử dụng các mô hình YOLOv8, MediaPipe, Dlib và phương pháp Perspective-n-Point (PnP) để phân tích hành vi tài xế theo thời gian thực trên nền kiến trúc client-server ba tầng. Kết quả thực nghiệm cho thấy hệ thống đạt độ chính xác phân loại trung bình 89,9% trên CPU phòng lab và khoảng 92% end-to-end trên GPU với tốc độ xử lý 20–30 FPS, độ trễ đầu cuối 100–200 ms, đáp ứng yêu cầu giám sát thời gian thực và hỗ trợ nâng cao an toàn lái xe.

Ghi chú: Bài báo là sản phẩm thuộc đề tài nghiên cứu khoa học sinh viên có mã đề tài: ĐTSV2526-280; Tên đề tài: “Nghiên cứu, phát triển hệ thống giám sát tài xế thời gian thực ứng dụng thị giác máy tính”.

Từ khóa: Phát hiện buồn ngủ, phân tâm khi lái xe, YOLOv8, MediaPipe, Dlib, an toàn giao thông, AI biên.

1. Giới thiệu

Tai nạn giao thông đường bộ vẫn là một trong những nguyên nhân hàng đầu gây tử vong và thiệt hại kinh tế trên toàn thế giới. Theo Tổ chức Y tế Thế giới (WHO), mỗi năm có khoảng 1,19 triệu người tử vong và hàng chục triệu người bị thương do tai nạn giao thông đường bộ [1], trong đó lỗi của người điều khiển phương tiện vẫn là nguyên nhân chủ yếu. Tại Việt Nam, theo báo cáo của Ủy ban An toàn giao thông Quốc gia, giai đoạn 2024–2025 cả nước ghi nhận hơn 6.000 trường hợp tử vong mỗi năm, trong đó tình trạng buồn ngủ, mất tập trung và sử dụng điện thoại khi lái xe là các yếu tố nguy cơ quan trọng dẫn đến nhiều vụ tai nạn nghiêm trọng [2, 18].

Bên cạnh đó, nhiều nghiên cứu đã chứng minh rằng trạng thái sinh lý và hành vi của người lái ảnh hưởng trực tiếp đến khả năng điều khiển phương tiện. Hiệp hội An toàn giao thông Hoa Kỳ (AAA Foundation for Traffic Safety) cho thấy người lái ngủ dưới 5 giờ trong vòng 24 giờ có nguy cơ gây tai nạn cao gấp 4,5 lần so với người ngủ đủ 7 giờ trở lên [3]. Tương tự, Cục Quản lý An toàn Giao thông Đường cao tốc Quốc gia Hoa Kỳ (NHTSA) cũng chỉ ra rằng tình trạng buồn ngủ và mất tập trung là nguyên nhân của hàng chục nghìn vụ tai nạn mỗi năm. Những thống kê này cho thấy nhu cầu cấp thiết đối với các hệ thống hỗ trợ lái xe có khả năng phát hiện sớm các hành vi nguy hiểm và đưa ra cảnh báo kịp thời.

Driver Monitoring System (DMS) là hướng nghiên cứu trọng tâm trong Intelligent Transportation Systems (ITS). Các phương pháp dùng cảm biến sinh lý (EEG, ECG) đạt độ chính xác cao nhưng bất tiện do yêu cầu đeo thiết bị [10]. Thị giác máy tính và học sâu đã mở ra giải pháp không xâm lấn dùng camera thông thường [11, 12]. Các nghiên cứu gần đây áp dụng EAR [4], head pose estimation [6, 14], YOLO [7, 8] để phát hiện buồn ngủ, mất tập trung, điện thoại và dây an toàn. Tuy nhiên, phần lớn chỉ tập trung vào một vài nhiệm vụ riêng lẻ, chưa có hệ thống tích hợp nhiều module giám sát tài xế trên cùng một pipeline thời gian thực.

Bài báo này đề xuất hệ thống giám sát tài xế thông minh tích hợp 6 module thị giác máy tính theo kiến trúc ba tầng truyền thống (tầng giao diện, tầng ứng dụng, tầng dữ liệu), trong đó tầng ứng dụng tích hợp một module AI suy luận chuyên biệt.

Các đóng góp chính:

- Kiến trúc AI thống nhất tích hợp 6 module giám sát tài xế (buồn ngủ, ngáp, tư thế đầu, điện thoại, dây an toàn, tay lái) trên cùng pipeline thời gian thực.
- Kết hợp Dlib, MediaPipe, YOLOv8 và PnP với chi phí tính toán thấp, phù hợp thiết bị biên.
- Cơ chế cảnh báo đa kênh (âm thanh, HTTP polling/SSE, MQTT) hỗ trợ người lái thời gian thực.

2. Các nghiên cứu liên quan

Có nhiều hành vi vi phạm của người lái xe có thể dẫn đến tai nạn giao thông, trong đó ba nhóm phổ biến nhất là buồn ngủ, tư thế lái không đúng quy cách và sử dụng vật thể lạ trong cabin. Đã có nhiều nghiên cứu trong và ngoài nước tập trung giải quyết từng nhóm hành vi này.

Phát hiện buồn ngủ là hướng nghiên cứu được quan tâm sớm nhất, với phương pháp kinh điển là sử dụng chỉ số PERCLOS và Eye Aspect Ratio (EAR). Soukupova và Cech [4] đã giới thiệu EAR được tính từ sáu điểm mốc quanh mắt sử dụng bộ dự đoán khuôn mặt 68 điểm của dlib [5]. Khi EAR giảm dưới ngưỡng trong thời gian liên tục, hệ thống xác định tài xế đang buồn ngủ. Các nghiên cứu khác [10, 13] sử dụng kết hợp nhiều chỉ số sinh lý để nâng cao độ chính xác.

Tư thế đầu không đúng quy cách cũng là một dấu hiệu cảnh báo mất tập trung quan trọng. Lawson và cộng sự [6] sử dụng phương pháp Perspective-n-Point (PnP) với mô hình khuôn mặt 3D để ước lượng ba góc Euler. Malek và cộng sự [14] phân loại tư thế đầu dựa trên landmark cho bài toán phục hồi chức năng. Phát hiện tư thế đầu bất thường ($\text{yaw} > 40^\circ$ hoặc $\text{pitch} > 35^\circ$)

được sử dụng làm tín hiệu cảnh báo mất tập trung.

Phát hiện vật thể lạ trong cabin (điện thoại, dây an toàn, tay lái) là bài toán được giải quyết chủ yếu bằng các mô hình phát hiện vật thể. Họ YOLO [7] đã chứng tỏ hiệu quả vượt trội trong phát hiện vật thể thời gian thực. Các phiên bản YOLOv8 [8] với kiến trúc CSPDarknet, PANet và head detection anchor-free cho phép đạt độ chính xác cao với tốc độ nhanh. Voulodimos và cộng sự [11] cung cấp tổng quan về học sâu trong thị giác máy tính. Shorten và Khoshgoftaar [16] khảo sát các kỹ thuật tăng cường dữ liệu ảnh cho học sâu.

Ở cấp độ hệ thống, các giải pháp thương mại như Seeing Machines [17], Smart Eye đã được triển khai trên xe hạng sang nhưng yêu cầu phần cứng chuyên dụng với chi phí cao. Xu hướng hiện nay là xây dựng các hệ thống DMS chi phí thấp trên nền tảng mã nguồn mở, phù hợp cho cả hậu mãi và tích hợp OEM. Nghiên cứu của Nimma và cộng sự [15] đề xuất mô hình Transformer-YOLOv8 cho giám sát video thời gian thực.

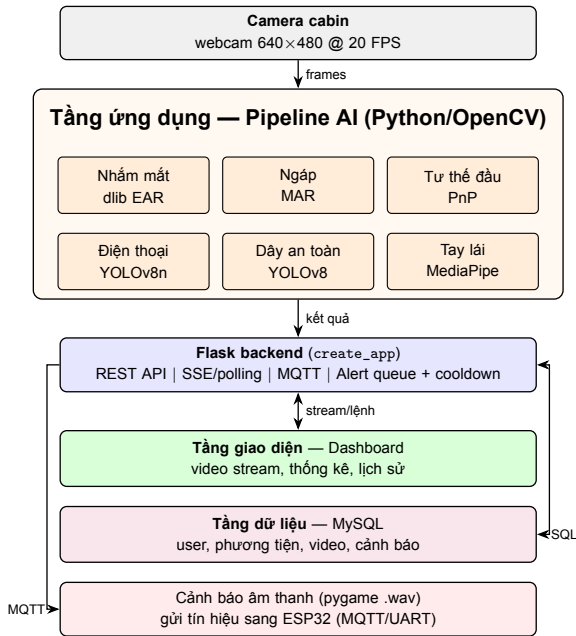
3. Phương pháp đề xuất

3.1. Kiến trúc tổng thể

Hệ thống được tổ chức theo kiến trúc ba tầng truyền thống (three-tier architecture), gồm tầng giao diện (Presentation Layer), tầng ứng dụng/xử lý nghiệp vụ (Application Layer) và tầng dữ liệu (Data Layer):

- Tầng giao diện (Presentation Layer, Next.js + Flask template): Dashboard giám sát, hiển thị video stream, thống kê cảnh báo và lịch sử vi phạm.
- Tầng ứng dụng/xử lý nghiệp vụ (Application Layer, Flask): Cung cấp REST API, SSE thời gian thực và HTTP polling, MQTT cho kết nối IoT; tiếp nhận kết quả suy luận, tổng hợp cảnh báo và cung cấp dữ liệu cho tầng giao diện.
- Tầng dữ liệu (Data Layer, MySQL): Lưu trữ thông tin người dùng, phương tiện, tuyến đường, kết quả phát hiện, video ghi hình và lịch sử cảnh báo.

Module AI suy luận (Python/OpenCV, YOLOv8, MediaPipe, Dlib, PnP) chỉ là một khối nhỏ thuộc tầng ứng dụng, chuyên xử lý video thời gian thực và phát hiện đa module (Hình 1). Trong cài đặt, module này nằm trong tiến trình backend (`backend/app/ai/`) và được khởi tạo cùng ứng dụng Flask qua `factory create_app`.



Hình 1: Sơ đồ kiến trúc ba tầng và pipeline AI với giao tiếp hai chiều

Về kỹ thuật triển khai, hệ thống áp dụng tải mô hình lười biếng (lazy loading) qua hàm `_load_once()` với khóa `model_lock` theo mẫu double-checked locking, giúp mô hình chỉ khởi tạo một lần tại lần gọi đầu tiên; đồng thời sử dụng đồng bộ hóa đa luồng (`threading.Lock`) cho hàng đợi cảnh báo và pipeline xử lý video.

3.2. Phát hiện nhắm mắt và buồn ngủ

Module sử dụng bộ phát hiện khuôn mặt dlib và bộ dữ liệu đoán điểm mốc 68 điểm [5]. Chỉ số EAR được tính cho từng mắt từ 6 điểm mốc theo phương pháp Soukupova và Cech [4] (index 36–41 cho mắt trái, 42–47 cho mắt phải):

$$EAR = \frac{\|p_2 - p_6\| + \|p_3 - p_5\|}{2\|p_1 - p_4\|} \quad (1)$$

Ngưỡng phát hiện nhắm mắt: $EAR < 0,30$ trong thời gian tối thiểu 2 giây liên tục.

3.3. Phát hiện ngáp

Phát hiện ngáp dựa trên khoảng cách giữa môi trên và môi dưới từ các điểm mốc miệng của bộ dữ liệu đoán 68 điểm [5], theo cùng ý tưởng đo khoảng cách landmark như EAR [4]. Ngáp được xem là chỉ báo buồn ngủ trong các tổng quan DMS [13]:

$$MAR = \frac{1}{|bottom|} \sum_{i \in bottom} y_i - \frac{1}{|top|} \sum_{j \in top} y_j \quad (2)$$

Trong đó $bottom = \{56, 57, 58, 65, 66\}$, $top = \{50, 51, 52, 61, 62\}$. Ngưỡng phát hiện: $MAR > 25$ pixel trong 15 khung hình liên tiếp (0,75 giây).

3.4. Ước lượng tư thế đầu

Hệ thống ước lượng tư thế đầu theo hướng PnP cho bài toán phát hiện mắt tập trung [6], kế thừa hướng theo dõi tư thế khuôn mặt cho giám sát tình tảo [10]. Phương pháp được hiện thực bằng hàm `cv2.solvePnP` với mô hình khuôn mặt 3D chuẩn nhân trắc học và sáu điểm mốc 2D từ bộ dữ liệu đoán 68 điểm [5] (đỉnh mũi, cằm, khóe mắt ngoài, khóe miệng). Phương trình chiếu phối cảnh:

$$s \cdot \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = K \cdot [R | t] \cdot \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix} \quad (3)$$

Từ ma trận xoay R , hệ thống trích xuất ba góc Euler (Yaw, Pitch, Roll). Do hành vi mắt tập trung chủ yếu thể hiện qua việc quay mặt sang hai bên hoặc cúi/ngửa đầu, ngưỡng cảnh báo trong cài đặt này được kích hoạt khi: $|\text{yaw}| > 40^\circ$ hoặc $\text{pitch} > 35^\circ$.

3.5. Theo dõi tay lái

Module sử dụng MediaPipe Hands [9] (21 landmarks, detection confidence 0,5) và MediaPipe Pose (33 landmarks) qua lớp `HandAndArmTracking`. Bộ phân loại nắm tay (fist classifier) kiểm tra 4 ngón (trỏ, giữa, áp út, út) dựa trên vị trí đầu ngón so với khớp ngón:

$$FistScore = \sum_{k \in \{8, 12, 16, 20\}} [\text{landmark}_k.y > \text{landmark}_{k-2}.y] \quad (4)$$

Các chỉ số: 8/12/16/20 (đầu ngón), 6/10/14/18 (khớp ngón). Ngón cái (4/3) được loại trừ. Nắm vô lăng được xác định khi $FistScore \geq 3$. Cảnh báo kích hoạt nếu không phát hiện nắm trong 3 giây liên tục, khoảng cách tối thiểu giữa các cảnh báo 5 giây.

3.6. Phát hiện sử dụng điện thoại

Mô hình YOLOv8n [8] thuộc họ YOLO [7] được fine-tuning trên 3.500 ảnh cabin kết hợp pretrained COCO (80 lớp) với freezing layers. Kết quả huấn luyện (Bảng 1) đạt $mAP@50 = 0,975$ và $mAP@50-95 = 0,748$ sau 100 epoch. Hình 2 cho thấy các chỉ số hội tụ ổn định từ epoch 50. Ma trận nhầm lẫn (Hình 3) xác nhận độ chính xác phân loại cao giữa lớp “cell phone” và nền. Hệ thống xác định hành vi sử dụng điện thoại khi độ tin cậy dự đoán vượt ngưỡng 0,5.



Hình 2: Đường cong huấn luyện mô hình phát hiện điện thoại



Hình 3: Ma trận nhầm lẫn của mô hình phát hiện điện thoại

3.7. Phát hiện dây an toàn

Module sử dụng YOLOv8 [8] được huấn luyện tùy chỉnh (trọng số `day_an_toan.pt`) trên tập dữ liệu 1.200 ảnh thu thập từ nhiều góc nhìn trong cabin, áp dụng transfer learning với fine-tuning. Kết quả (Bảng 1) đạt $mAP@50 = 0,753$ và $mAP@50-95 = 0,354$ sau 50 epoch (Hình 4). Ma trận nhầm lẫn (Hình 5) cho thấy lớp “seat_belt” đạt 0,73, thấp hơn rõ rệt so với lớp “cell phone” (0,98), phản ánh tập dữ liệu nhỏ hơn và biến động góc nhìn lớn. Độ chính xác thấp hơn so với mô hình điện thoại do kích thước tập dữ liệu nhỏ hơn và sự đa dạng về tư thế, góc nhìn của dây an toàn.



Hình 4: Đường cong huấn luyện mô hình phát hiện dây an toàn (50 epoch)



Hình 5: Ma trận nhầm lẫn của mô hình phát hiện dây an toàn

Bảng 1: Kết quả huấn luyện các mô hình YOLO

Mô hình	Dữ liệu	Số ảnh	$mAP@50$	$mAP@50-95$
CellPhone-YOLOv8n	COCO + In-Cabin Real	3.500	0,975	0,748
Seatbelt-YOLOv8	Custom In-Cabin	1.200	0,753	0,354

3.8. Hệ thống giao tiếp hai chiều AI-Web

Luồng giao tiếp hai chiều (two-way communication) được tổ chức giữa ba tầng chuẩn: module AI suy luận (thuộc tầng ứng dụng) xử lý video với cơ chế tải mô hình lười biếng an toàn đa luồng; backend Flask ở tầng ứng dụng cung cấp REST API, HTTP polling/SSE và MQTT cho kết nối IoT; dashboard web ở tầng giao diện thực hiện giám sát, hiển thị video stream, thống kê cảnh báo và lịch sử vi phạm. Cơ chế two-way communication cho phép hệ thống gửi cảnh báo, video stream đến web và web gửi lệnh bật/tắt module, điều chỉnh ngưỡng đến hệ thống.

Pipeline xử lý AI hoạt động tuần tự qua các module

phát hiện. Mỗi module có thể bật/tắt độc lập. Cơ chế tổng hợp cảnh báo đa module theo mức ưu tiên với thời gian cooldown riêng (Bảng 3). Cảnh báo được truyền qua ba kênh: âm thanh (pygame), hiển thị trên video stream, và thời gian thực qua HTTP polling/SSE và MQTT.

4. Thử nghiệm và kết quả

4.1. Thiết lập thử nghiệm

Hệ thống được kiểm thử trên hai cấu hình: GPU (Intel Core i7-12700H, NVIDIA RTX 3050 Ti) và CPU (Intel Core i7-1165G7, Intel Iris Xe). Camera webcam độ phân giải 640×480 ở 20 FPS, YOLOv8 chạy đầu vào 640×640 . Đánh giá gồm ba mức tách biệt: (1) chỉ số $mAP@50/mAP@50-95$ trên tập huấn luyện (Bảng 1: module điện thoại dùng 3.500 ảnh trích xuất từ COCO kết hợp cabin thực tế, module dây an toàn dùng 1.200 ảnh cabin tùy chỉnh); (2) kiểm thử chức năng từng module trong phòng lab trên CPU với video cabin (độ chính xác phân loại, Bảng 4); (3) kiểm thử toàn hệ thống end-to-end trên luồng webcam trực tiếp (độ chính xác cảnh báo và FPS/độ trễ, Bảng 5). Dữ liệu cabin được thu thập tại Thư viện Trường Đại học Công nghệ Giao thông vận tải từ tháng 8/2025 đến tháng 2/2026; vận hành thử end-to-end được thực hiện trên cabin xe của cộng tác viên trong tháng 3/2026. Thời gian suy luận được đo bằng cơ chế timing instrumentation ghi log mỗi 100 frame.

4.2. Kết quả và đánh giá

Hệ thống có cơ chế đo thời gian suy luận (timing instrumentation) ghi log mỗi 100 frame. Bảng 2 liệt kê các tham số ngưỡng phát hiện của từng module.

Việc lựa chọn ngưỡng ảnh hưởng trực tiếp đến độ chính xác và tốc độ phát hiện. Ngưỡng EAR = 0,30 kết hợp với thời gian nhắm mắt tối thiểu 2 giây giúp phân biệt chớp mắt tự nhiên (100–400 ms) với buồn ngủ thực sự, giảm dương tính giả. Ngưỡng YAWN = 25 pixel và 15 khung hình liên tiếp (tương đương 0,75 giây ở 20 FPS) loại bỏ các chuyển động miệng ngắn (nói chuyện, cười). Ngưỡng tư thế đầu $|yaw| > 40^\circ$ và $pitch > 35^\circ$ theo biên độ nhìn tự nhiên khi lái xe [6]; $conf > 0,5$ cho YOLO để cân bằng precision-recall; $FistScore \geq 3$ (cơ chế đa số với 4/5 ngón) cân bằng giữa độ nhạy và độ đặc hiệu. Cơ chế cooldown (2–4 giây) ngăn cảnh báo trùng lặp, giảm nhiễu cho người lái.

Bảng 2: Tham số ngưỡng phát hiện các module

Module	Tham số	Giá trị
Phát hiện nhắm mắt	EAR_THRESHOLD	0,30
	EAR_MIN_DURATION	2 giây
Phát hiện ngáp	YAWN_THRESHOLD	25 pixel
	YAWN_CONSEC_FRAMES	15 khung hình
Tư thế đầu	Ngưỡng yaw	$> 40^\circ$
	Ngưỡng pitch	$> 35^\circ$
Điện thoại	Confidence	$> 0,5$
Dây an toàn	Confidence	$> 0,5$
Tay lái	warning_duration	3 giây
	FistScore	≥ 3
Cooldown	eye/yawn/head/phone	2–3 giây
	seatbelt/hand	4 giây

Bảng 3: Mức ưu tiên và hành động xử lý cảnh báo

Module	Mức	Âm thanh	Kênh truyền
Nhắm mắt	critical	nham_mat.wav	HTTP polling + MQTT
Điện thoại	critical	not_phone.wav	HTTP polling + MQTT
Dây an toàn	critical	seatbelt_alert.wav	HTTP polling + MQTT
Ngáp	warning	ngap_ngu.wav	HTTP polling
Mất tập trung	warning	dau_quay.wav	HTTP polling
Tay lái	warning	tay_lai_xe.wav	HTTP polling

Bảng 4 trình bày thời gian suy luận và độ chính xác phân loại của từng module trong kiểm thử lab trên CPU với độ phân giải 640×480 . Pipeline xử lý tuần tự trên CPU đạt 2,16 FPS (462,37 ms/frame), trong đó các mô hình YOLOv8 chiếm phần lớn thời gian xử lý.

Bảng 4: Kết quả kiểm thử chức năng từng module trong phòng lab trên CPU.

Module	Thời gian (ms)	FPS	Độ chính xác phân loại lab (%)
Phát hiện khuôn mặt và điểm mốc (dlib)	51,96	19,24	95,0
Phát hiện điện thoại (YOLOv8n-custom)	173,23	5,77	99,8
Phát hiện dây an toàn (YOLOv8-custom)	153,92	6,49	80,7
Ước lượng tư thế (MediaPipe)	83,26	12,01	90,5
Toàn bộ hệ thống	462,37	2,16	89,9

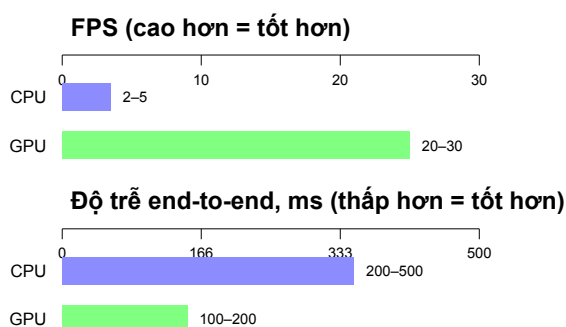
Độ chính xác của từng module chịu ảnh hưởng trực tiếp từ ngưỡng phát hiện. dlib đạt 95,0% nhờ HOG+SVM phát hiện khuôn mặt chính diện ổn định [5], nhưng giảm khi góc nghiêng vượt 30° . YOLOv8n phát hiện điện thoại đạt precision 99,8% ở ngưỡng conf $> 0,5$ trên tập kiểm thử cô lập – đây là độ chính xác phân loại trong điều kiện thử nghiệm chuẩn. Khi triển khai trên luồng stream thực tế trong cabin, module vẫn duy trì độ nhạy cao và góp phần đưa độ chính xác cảnh báo end-to-end toàn hệ thống đạt $\sim 92\%$ trên GPU, dù trong một số tình huống đặc thù góc nhìn bị khuất tay cầm hoặc phản xạ ánh sáng mạnh có thể làm giảm nhẹ độ tin cậy tức thời. YOLOv8 phát hiện dây an toàn đạt 80,7% với conf $> 0,5$, thấp hơn do tập huấn luyện chỉ 1.200 ảnh. MediaPipe đạt 90,5% nhờ mô hình TFLite nhẹ hoạt động ổn định trong cabin [9]. Giá trị trung bình 89,9% trong bảng này là kết quả lab trên CPU, phản ánh sự cân bằng giữa độ nhạy và độ đặc hiệu ở các ngưỡng đã chọn. Về thời gian, hai mô hình YOLOv8 chiếm $\sim 71\%$ tổng thời gian xử lý (327 ms), cho thấy

đây là nút thắt cổ chai cần tối ưu.

Trên luồng webcam trực tiếp, hệ thống đạt tốc độ 20–30 FPS với độ trễ 100–200 ms và độ chính xác end-to-end khoảng 92% trên GPU, trong khi trên CPU chỉ đạt 2–5 FPS với độ trễ 200–500 ms và khoảng 85% (chênh lệch do pipeline tuần tự trên CPU bỏ lỡ sự kiện khi tải YOLO cao, Bảng 5, Hình 6). GPU nhanh hơn 6–10 lần, khẳng định vai trò thiết yếu của tăng tốc phần cứng trong các hệ thống DMS đa module.

Bảng 5: Kết quả vận hành thử hệ thống hoàn chỉnh trên webcam trực tiếp sau khi tích hợp toàn bộ

Cấu hình	FPS	Độ trễ (ms)	Độ chính xác end-to-end
CPU (i7-1165G7, Iris Xe)	2–5	200–500	$\sim 85\%$
GPU (i7-12700H, RTX 3050 Ti)	20–30	100–200	$\sim 92\%$

**Hình 6:** So sánh FPS và độ trễ end-to-end giữa CPU và GPU (thanh bar vẽ theo giá trị giữa khoảng trong Bảng 5)**Bảng 6:** Danh mục kiểm thử chức năng hệ thống (kiểm thử thủ công, chưa có bộ test tự động)

Chức năng	Kết quả thử công
Hiển thị video stream trên dashboard	Đạt
Bật/tắt từng module AI từ web	Đạt
Điều chỉnh ngưỡng phát hiện từ web	Đạt
Cảnh báo âm thanh pygame theo mức ưu tiên	Đạt
Nhận cảnh báo qua HTTP polling/SSE	Đạt
Đẩy cảnh báo qua MQTT tới ESP32	Đạt khi có broker
Lưu lịch sử cảnh báo và video vào MySQL	Đạt

5. Thảo luận

Kết quả cho thấy thách thức chính không nằm ở từng thuật toán riêng lẻ mà ở tích hợp đa module trên cùng pipeline thời gian thực. Hai mô hình YOLOv8 chiếm $\sim 71\%$ thời gian xử lý (327 ms trên CPU), khiến pipeline tuần tự chỉ đạt 2–5 FPS và bỏ lỡ sự kiện; trên GPU hệ thống đạt 20–30 FPS, khẳng định tăng tốc phần cứng là điều kiện bắt buộc với DMS đa module. Chênh lệch độ chính xác giữa điện thoại (99,8%) và dây an toàn (80,7%) phản ánh trực tiếp quy mô và đa dạng dữ liệu (3.500 ảnh COCO kết hợp cabin so với 1.200 ảnh cabin), cùng biến động ánh sáng và góc nhìn trong cabin [11, 12]. Các ngưỡng EAR, MAR, yaw/pitch, conf và FistScore cùng cơ chế cooldown thực chất là điểm cân bằng giữa độ nhạy và độ đặc hiệu: siết ngưỡng

giảm báo giả nhưng tăng bỏ sót ngoài cabin thực tế.

Kết quả thực nghiệm cho thấy hệ thống đáp ứng tốt các mục tiêu về chức năng và hiệu năng. Hệ thống đạt tốc độ xử lý 20–30 FPS trên GPU, đáp ứng yêu cầu giám sát thời gian thực. Tuy nhiên, một số hạn chế cần được cải thiện:

- Hiệu năng trên CPU còn thấp (2–5 FPS) do pipeline tuần tự chưa tận dụng đa luồng.
- Độ chính xác phát hiện dây an toàn (80,7%) cần cải thiện thông qua mở rộng tập dữ liệu.
- Chưa có đánh giá trên bộ dữ liệu chuẩn công khai như Drive&Act.
- Giao diện web hiện cập nhật trạng thái theo cơ chế polling định kỳ; có thể chuyển sang WebSocket để tăng tính thời gian thực.

6. Kết luận và hướng phát triển

Bài báo đã trình bày một hệ thống giám sát tài xế thông minh ứng dụng AI với kiến trúc đa module tích hợp sáu thành phần phát hiện hành vi tài xế: buồn ngủ (dlib EAR), ngáp (facial landmarks), tư thế đầu (PnP), điện thoại (YOLOv8n), dây an toàn (YOLOv8 custom) và tay lái (MediaPipe). Hệ thống được tổ chức theo kiến trúc ba tầng truyền thống (giao diện–ứng dụng–dữ liệu), trong đó tầng ứng dụng tích hợp module AI suy luận chuyên biệt với cơ chế tải mô hình luân phiên an toàn đa luồng, hàng đợi cảnh báo và cảnh báo đa kênh (âm thanh, HTTP polling/SSE, MQTT).

Kết quả thực nghiệm trên GPU (RTX 3050 Ti) cho thấy hệ thống đáp ứng yêu cầu giám sát thời gian thực với tốc độ xử lý 20–30 FPS, độ trễ đầu cuối 100–200 ms.

Hướng phát triển trong tương lai:

- Triển khai Edge AI: Chuyển pipeline lên NVIDIA Jetson Nano với TensorRT để tối ưu inference, giảm chi phí phần cứng.
- Mô hình tiên tiến: Nâng cấp lên YOLOv10/v11, áp dụng Transformer cho phân tích chuỗi hành vi phức tạp [15].
- Mở rộng dữ liệu: Thu thập thêm dữ liệu đa dạng về điều kiện ánh sáng, góc nhìn và đối tượng người lái.
- Tối ưu pipeline: Chuyển sang xử lý đa luồng và pipeline bất đồng bộ.

Lời cảm ơn

Nghiên cứu này được tài trợ bởi Trường Đại học Công nghệ Giao thông vận tải (UTT), mã số đề tài: ĐTSV2526-280. Nhóm tác giả chân thành cảm ơn các cộng tác viên đã hỗ trợ thu thập dữ liệu và đánh giá thử nghiệm hệ thống.

Tài liệu tham khảo

- [1] World Health Organization. (2023). *Global status report on road safety 2023*. Geneva: WHO.
- [2] Hiến, Đ. (2026). *Báo cáo ATGT Quốc gia giai đoạn 2024–2025*. Ủy ban An toàn giao thông Quốc gia.
- [3] AAA Foundation for Traffic Safety. (2018). Acute sleep deprivation and risk of motor vehicle crash involvement. *AAA NewsRoom*.
- [4] Soukupova, T., & Cech, J. (2016). Eye blink detection using facial landmark tracking. In *Proceedings of the International Conference on Computer Vision Theory and Applications (VISAPP)* (Vol. 2, pp. 328–335).
- [5] King, D. E. (2009). Dlib-ml: A machine learning toolkit. *Journal of Machine Learning Research*, 10, 1755–1758.
- [6] Lawson, A., Kim, J., & Lee, T. (2020). Head pose estimation for driver distraction detection. *IEEE Transactions on Intelligent Transportation Systems*, 21(7), 2964–2973.
- [7] Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You only look once: Unified, real-time object detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (pp. 779–788).
- [8] Jocher, G., Chaurasia, A., & Qiu, J. (2023). *YOLOv8 by Ultralytics* [Computer software]. GitHub.
- [9] Lugaresi, C., et al. (2019). MediaPipe: A framework for building perception pipelines. *arXiv preprint arXiv:1906.08172*.
- [10] Ji, Q., & Yang, X. (2002). Real-time eye, gaze, and face pose tracking for monitoring driver vigilance. *Real-Time Imaging*, 8(5), 357–377.
- [11] Voulodimos, A., et al. (2018). Deep learning for computer vision: A brief review. *Computational Intelligence and Neuroscience*, 2018.
- [12] LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. *Nature*, 521(7553), 436–444.
- [13] Ngxande, M., Tapamo, J., & Burke, M. (2017). Driver drowsiness detection: A review. In *Proceedings of the IEEE AFRICON Conference* (pp. 991–996).
- [14] Malek, S., et al. (2021). Head pose estimation using facial-landmarks classification for rehabilitation games. *Pattern Recognition Letters*.
- [15] Nimma, D., et al. (2025). Object detection in real-time video surveillance using attention based Transformer-YOLOv8 model.
- [16] Shorten, C., & Khoshgoftaar, T. (2019). A survey on image data augmentation for deep learning. *Journal of Big Data*, 6(1), 1–48.

[17] Seeing Machines. (2022). *Guardian DMS: Driver Monitoring System OEM* [Technical documentation].

[18] Báo Giao thông. (2025). Thống kê tai nạn giao thông tại Việt Nam. *Báo Giao thông*.